

Introducción a los Lenguajes de Programación

UNTDF – Sede Ushuaia
2024

Si bien hay varios libros en el programa, el texto principal en la materia es el libro:

Robert Sebesta: Concepts of Programming Languages

Conceptos básicos de LPs

*Una descripción completa de los lenguajes de programación incluye el **modelo computacional**, la **sintaxis** y **semántica** de programas, y las consideraciones pragmáticas que dan forma a los lenguajes. (Aaby, A.)*

Conceptos básicos de LPs

- *Qué es un Lenguaje?*

- ✓ Un **cómputo** es un proceso que consiste en aplicar una serie de **operaciones** estructuradas a un conjunto de valores o datos de entrada, con el objetivo de obtener nuevos valores o datos como resultado.
- ✓ Un **programa** es una colección definida y ordenada de **cómputos** diseñada para realizar una tarea específica o resolver un problema concreto. Es la expresión de uno o más **cómputos** escritos de manera que una máquina pueda ejecutarlos..
- ✓ Un **lenguaje de programación** es un conjunto de reglas **sintácticas** y **semánticas** usadas para definir programas. Este sistema de notación está compuesto por instrucciones que son comprendidas y ejecutadas por máquinas, usualmente computadoras..

Conceptos básicos de LPs

- *Qué es un Lenguaje?*

- ✓ La **sintaxis** de un lenguaje de programación es el conjunto de reglas que define la estructura y la forma correcta de escribir programas en ese lenguaje. Esto incluye la disposición correcta de palabras clave, símbolos, y estructuras de control para formar instrucciones válidas.
- ✓ La **semántica** de un lenguaje de programación es el significado de las construcciones sintácticas y las operaciones definidas dentro del lenguaje. Describe cómo las instrucciones sintácticamente correctas interactúan con el modelo computacional para producir un comportamiento o resultado esperado en la máquina.

Conceptos básicos de LPs

- *Qué es un Lenguaje?*

- ✓ Los lenguajes de programación aparecen en la década del 50.
- ✓ Se busca poder **expresivo**, **simplicidad**, **eficiencia**.
- ✓ Un Lenguaje de Programación está basado en tres conceptos fundamentales:
 - Modelos computacionales
 - Datos y Estructuras de Datos
 - Abstracción y generalización

Sintaxis y Semántica

- Cualquier lenguaje de programación especifica un *conjunto de reglas* para la forma de programas válidos en ese lenguaje.
- Las **reglas de sintaxis del lenguaje** establecen cómo formar **expresiones, declaraciones y programas que se vean correctos**. Por ejemplo, en algunos lenguajes se debe escribir un “;” al final de cada línea.
- Las **reglas semánticas del lenguaje**, nos dicen cómo construir expresiones, declaraciones y programas con significado. Por ejemplo, antes de usar una variable en alguna expresión, debe declararse la variable.

Lenguajes naturales vs. LPs

- **Parecidos:**

Ambos se articulan sobre una **sintaxis** y una **semántica**

- **Diferencias:**

- a) Amplitud, expresividad, sutileza de los lenguajes naturales frente a los computacionales.
- b) Los lenguajes de programación se diseñan con intención de implementarse sobre máquinas.

Importancia del Estudio de Lenguajes

- **Capacidad de expresión de ideas**

- ✓ La habilidad y **capacidad de comprensión** en un lenguaje afecta a nuestra **capacidad para pensar y formar ideas. La misma restricción sufren los programadores.** El lenguaje con el cual trabajan limita las abstracciones que pueden realizar.
- ✓ Una comprensión profunda de los LP puede **incrementar nuestra capacidad para pensar aproximaciones distintas a problemas**
- ✓ **Aprender nuevas construcciones** de lenguajes ayuda a reducir las limitaciones que el uso de un solo tipo de lenguajes provoca en los programadores.
- ✓ El estudio de conceptos de lenguajes de programación **permite conocer las características importantes de los distintos lenguajes y a usarlas.**

Importancia del Estudio de Lenguajes

- **Uso más eficiente de un LP**

- ✓ Una mejor **comprensión de la función e implementación de las estructuras** de los LP permite un uso más eficiente del Lenguaje.

- **Background para la selección del lenguaje apropiado**

- ✓ Muchos programadores utilizan en su trabajo diario y/o recibieron educación formal en uno o dos lenguajes. Para nuevos proyectos continúan utilizando el LP que conocen, aún cuando **existen otros más adecuados al requerimiento**.
- ✓ El conocimiento de otros lenguajes le permitiría una **mejor selección del lenguaje adecuado al problema**.
- ✓ El conocimiento de las características básicas de **fortalezas y debilidades de cada LP**, proporciona una selección más amplia de alternativas.

Importancia del Estudio de Lenguajes

- **Habilidad para el aprendizaje de nuevos lenguajes**

- ✓ La Ingeniería de software es una disciplina en continua evolución, lo cual involucra un **aprendizaje continuo**.
- ✓ El proceso de aprendizaje de un nuevo lenguaje de programación es un proceso lento y difícil, cuando se conocen sólo uno o dos LPs.
- ✓ Una vez adquiridos los conceptos básicos de LPs, **se vuelve más sencillo el aprendizaje de nuevos lenguajes y se facilita la lectura y comprensión de manuales y literatura relativa a lenguajes y compiladores.**

Importancia del Estudio de Lenguajes

- **Mejor comprensión del significado de la implementación.**
 - ✓ Conocer **aspectos de implementación del lenguaje, permite utilizarlo de forma más inteligente,** comprendiendo las opciones entre construcciones del lenguaje y las consecuencias de esa selección.
 - ✓ Pueden **detectarse y comprenderse cierto tipo de fallas y visualizar alternativas más eficientes para el desarrollo de un programa.**
- **Habilidad para el desarrollo de nuevos lenguajes.**
Lenguajes Específicos de Dominio.

Principios básicos de LPs

Principio de completitud computacional

El modelo computacional para lenguajes de programación de propósitos generales debe ser universal.

Principio de Implementación

La implementación debe ser eficiente en el uso de espacio y tiempo.

Principio de Programación

Un lenguaje de programación debe diseñarse para facilitar **legibilidad y escrito-capacidad** para sus usuarios humanos y la ejecución eficaz en el hardware disponible.

El programa debe escribirse en un lenguaje que refleje el dominio del problema.

Principio de abstracción

No repetir algo más de una vez.

Principios básicos de LPs

Principio de estructura aparente:

El comportamiento del programa concuerda con su estructura sintáctica o código fuente

Principio de simplicidad:

Simple de entender y de programar en el lenguaje.

Principio de seguridad:

Detección de errores en diferentes niveles del lenguaje.

Principio de regularidad:

Reglas regulares, sin excepciones, son fáciles de aprender, usar, describir e implementar.

Atributos de un buen LPs

Simplicidad.

Debe haber tan pocos conceptos básicos como sea posible.

Uniformidad.

Los conceptos básicos deben aplicarse de forma consistente y universalmente. Debemos poder usar las características del lenguaje en contextos diferentes sin cambiar su formato.

Claridad.

Deben definirse bien los mecanismos, y el resultado de código debe ser fácilmente predecible. Por ej., en C confusión entre el operador de la asignación (=) y el de la igualdad (==).

Atributos de un buen LPs

Abstracción

La abstracción generalmente significa ocultar **detalles** construyendo una caja alrededor de ellos y permitiendo sólo inspección limitada de sus contenidos.

Ocultamiento de la información

Las unidades del programa **sólo deben tener acceso a la información** que ellos requieren.

Modularidad

Las interfaces entre unidades de programación deben declararse explícitamente.

Atributos de un buen LPs

Seguridad

Los **errores** semánticos deben ser perceptibles, preferentemente en tiempo de **compilación**.

Posibilidad de tratar excepciones

Permitir el tratamiento de posibles sucesos no deseados (desbordamiento, entradas no válidas, etc.) y la posibilidad de dar respuestas adecuadas a estos sucesos.

Expresividad

Es una medida de la naturalidad con que se puede representar la estrategia de solución de un problema mediante una estructura del lenguaje.

Java incorpora cada vez mas estructuras de datos y expresiones sintácticas que dan riqueza y permiten al programador buscar la mejor forma de expresar una solución.

Atributos de un buen LPs

Legibilidad o facilidad de lectura

Posibilidad de **seguir la lógica del programa y descubrir la presencia de errores**, mediante un examen del mismo.
Sencillez, claridad y coherencia.

Facilidad de escritura

Posibilidad de **expresar un programa en una forma que sea natural para el problema**. El programador no debe distraerse con detalles y trucos del lenguaje.

Facilidad de mantenimiento

Características que **mejoran la modificabilidad de un programa**, como nombres simbólicos a constantes, **modularización**, **encapsulamiento**, **portabilidad**, mecanismos para la **abstracción**, etc.

Atributos de un buen LPs

Facilidad para verificar programas

La confiabilidad de los programas escritos en un LP es una preocupación medular. **La verificación puede seguir métodos formales o no.** La sencillez de escritura semántica y sintáctica ayudan a simplificar la verificación.

Portabilidad

Un lenguaje que está ampliamente disponible y cuya definición es independiente de una máquina particular constituye una base útil para la producción de programas transportables.

Eficiencia

La eficiencia no sólo se mide por la velocidad de ejecución y por el espacio ocupado por el programa. **El esfuerzo inicial en el desarrollo y el necesario para su mantenimiento posterior deben tenerse en cuenta.**

Contribuyen: la facilidad de escritura, la mantenibilidad y la **optimización automática de código.**

Entorno de programación

Editores especiales, paquetes de prueba, ayudas.

Factores que influyen sobre el diseño de un LP

Arquitectura de computadoras

Se basaron en Von Newman: imperativos

Datos y programas en memoria

Posteriormente: lenguajes funcionales

Metodologías de programación

Hasta principios de 70 " artesanal

Desde 70: Análisis y Diseño

Programación estructurada

Abstracción (Simula, Ada, etc.)

Programación Funcional (LISP)

Orientación a Objetos (smalltalk)

Multiparadigmas (python)

Lenguajes y Paradigmas

- Formas de **conceptualizar la estructuración y organización** de cómo se llevan a cabo las tareas en una computadora.
- Un paradigma de programación es una **colección de modelos conceptuales** que juntos modelan el proceso de diseño y determinan la estructura de un programa.
- Podemos definir un paradigma de LPs como un estilo, tipo o familia de lenguajes de programación.
- Se pueden definir paradigmas:
 - de acuerdo al modelo computacional
 - a la forma de organizar los programas.

Lenguajes y Paradigmas

Paradigmas más importantes en los que podemos clasificar los lenguajes

- Programación Imperativa. (C, C++, Go, JAVA)
- Programación Orientada a Objetos (Smalltalk, JAVA, Dart)

Paradigma Declarativo

- Programación Lógica (declarativo) (Prolog)
- Programación Funcional (Lisp, Scheme, Haskell, Clojure, Plutus)

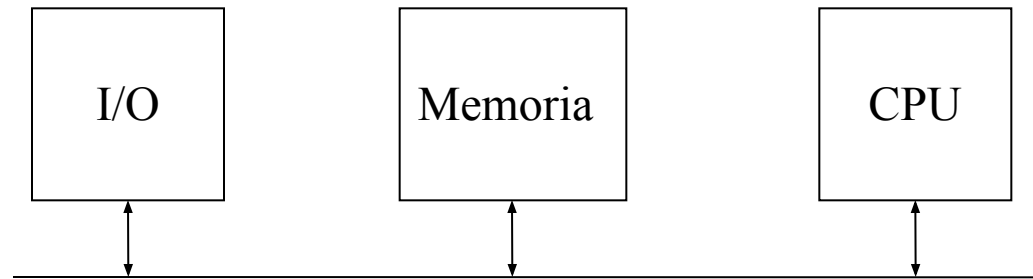
• Multiparadigmas

- Programación OOP y Funcional (Scala, Java, F#, Python)

Modelo computacional –LPs y Arquitectura

- La arquitectura de las computadoras tiene influencia sobre los lenguajes, dado que plantea la restricción al lenguaje acerca de lo que puede ser implementado en forma eficiente en las máquinas.
- Los lenguajes convencionales han sido desarrollados sobre la base de las ideas de Von Neumann, dado que la mayoría de las computadoras soporta tal arquitectura.
- La **arquitectura de Von Neumann** se basa en la idea de:
 - una memoria que contiene instrucciones y datos,
 - una CPU y
 - una unidad I/O

Modelo computacional –LPs y Arquitectura



- La CPU es responsable de tomar instrucciones de la memoria una por vez.
- Las instrucciones de máquina se toman de la memoria y se manipulan vía **operaciones aritméticas o lógicas en la CPU**; los resultados se vuelven a copiar en las celdas de memoria.
- La ejecución de las instrucciones resulta en *cambios de estado* de la máquina, lo cual se representa por los contenidos de la memoria.

Paradigma Imperativo

- *Lenguajes imperativos.*

- lenguajes convencionales **basados en el modelo computacional de Von Neumann**,
- pueden verse como **abstracciones** de la arquitectura subyacente de Von Neumann.
- retienen como modelo computacional dicha arquitectura, pero realizan abstracción de los pasos individuales de ejecución.

El *modelo imperativo de cómputo* está **basado en la perspectiva del procesador** ⇒ Consiste de:

- ✓ *la ejecución secuencial de instrucciones paso a paso,*
- ✓ *un conjunto de valores incluyendo un estado y*
- ✓ *la operación de asignación para modificar el estado*

Paradigma Imperativo

- **Clara influencia del hardware:**

- memoria $\Rightarrow$ variables de programa
- saltos condicional e incondicional $\Rightarrow$ sentencias de control
- transferencia de datos entre memoria y CPU $\Rightarrow$ instrucción de asignación.

- La ejecución secuencial de instrucciones de los lenguajes refleja el **ciclo de búsqueda y ejecución secuencial** de las instrucciones ejecutadas por el procesador.
- El estado es el conjunto de pares nombre-valor de las constantes y variables.
- Las **variables** de los lenguajes imperativos, que pueden ser modificadas por instrucciones de asignación, **reflejan el comportamiento de las celdas de memoria** de la arquitectura del hardware.
- Los programas consisten en **sucesiones de asignaciones y un cómputo es una sucesión de estados.**

Paradigma Imperativo. Desarrollo histórico

- Históricamente se han desarrollado en niveles crecientes de abstracción.
- En los primeros tiempos de la computación, los lenguajes **assembler** se inventaron para proveer formas primitivas de abstracción.
- Posteriormente los diseñadores de lenguajes desarrollaron otro tipo de abstracciones, tales como **procedimientos y funciones, tipos de datos**, etc.
- Cuando la programación imperativa se combina con **subprogramas**, se la denomina **programación procedural**.
- **Características de un programa en lenguaje Imperativo**
 - Variables
 - Sistema de tipos
 - Operadores
 - Asignación
 - Flujo de control

Paradigma Declarativo

- Algunos lenguajes, abandonaron el modelo **computacional de Von Neumann**.
- En un programa en LP declarativo **se describen las estructuras de datos, las relaciones entre ellos y un objetivo**.
- El proceso por el que se ejecuta dicha tarea no se especifica de forma explícita, sino que lo determina el sistema de traducción del lenguaje.
- Dentro del paradigma declarativo encontramos dos corrientes o familias de lenguajes *basados en fundamentos matemáticos*, en lugar de la tecnología del hardware subyacente: **la teoría de funciones recursivas y de la lógica matemática**.
- Respectivamente son: **Programación funcional y Programación lógica**

Paradigma Declarativo - Objetivos

- Poder olvidar cómo es la máquina en la que se trabaja
- Poder olvidar las operaciones concretas de la misma
- Concentrarse en las características del **problema**.
- Los lenguajes declarativos proporcionan una **alta capacidad de abstracción**.
- El programador centra su preocupación en el qué, dejando en segundo plano el cómo. Se describe el problema en términos abstractos.

Paradigma Declarativo - Funcional

- *Se emplean funciones al estilo de las funciones matemáticas para representar el problema a resolver.*
- Las funciones indican cómo obtener un resultado a partir de unos argumentos.
- El paradigma funcional esta basado en el modelo matemático de **composición funcional**.
- En este modelo **el resultado de un cálculo es la entrada del siguiente y así sucesivamente hasta que una composición produce el resultado deseado.**
- **No existe el concepto de celda de memoria que es asignada o modificada.**
- **Tampoco existen sentencias imperativasl.**

Paradigma Declarativo - Funcional

- Incorpora el concepto de *función como objeto de primera clase*:
 - las funciones pueden ser tratadas como datos, pueden pasar como parámetros, calculadas y devueltas como valores normales, y mezcladas en el cálculo como otras formas de datos.
 - En este paradigma, la solución de un problema se concibe como una **composición de funciones**.
 - Por ejemplo para ordenar una lista se puede diseñar la solución como una concatenación de listas mas pequeñas, cada una de las cuales ya esta ordenada.
 - Esto reduce el problema a seleccionar las listas más pequeñas.

Paradigma Declarativo - Funcional

La programación funcional está basada en el concepto matemático de función

Los lenguajes de programación funcionales incluyen:

- **Un conjunto de funciones primitivas.**
- Un conjunto de formas funcionales. **Por ejemplo, la composición.**
- **La operación de aplicación.**
- Un conjunto de objetos de datos y las funciones asociadas.
- Un mecanismo para ligar un nombre a una función.

Algunos lenguajes que implementan este paradigma:

LISP, FP, Scheme, ML, Miranda, Haskell, Clojure

Paradigma Declarativo - Funcional

- Un programa funcional viene definido por la ecuación:
Funciones + estructuras de datos = programas
- Los elementos fundamentales son:
 - **Funciones:** son reglas de correspondencia o asociación entre miembros de un conjunto (dominio) y miembros de otro conjunto (rango).
 - **Definición de funciones:** especifica el dominio, el rango y la regla de correspondencia.
 - **Aplicación de funciones:** se obtiene el resultado, el rango que se corresponde con los elementos del dominio para la cual se está aplicando.
 - **Recursividad:** la iteración con asignación no destructiva. El carácter recursivo de esta programación requiere de este mecanismo, especialmente utilizado en la construcción de funciones.
 - **Listas:** son las estructuras básicas de almacenamiento de la información

Paradigma Declarativo - Funcional

- La ejecución del programa funcional consiste en el **cálculo del valor asociado al dato de entrada** de acuerdo con las definiciones dadas para las funciones en el programa.
- El proceso de cálculo de dicho valor se conoce como evaluación del dato de entrada.
- Las **funciones son valores, al igual que los números, las tuplas, etc.**
 - pueden ser argumento de otras funciones
 - pueden ser resultado de otras funciones
 - pueden almacenarse en estructuras de datos
 - pueden ser estructuras de datos

Paradigma Declarativo - Funcional

Programa: definición de una función, descrita en una notación basada en principios matemáticos, que permite resolver un problema dado.

*Un **programa** es una colección de definiciones de funciones y un **cómputo** es la aplicación de la función (la evaluación de una expresión).*

Propiedades

- **No existe la noción variable en el sentido de la programación imperativa.**
- Las funciones son objetos de datos de primera clase
- Manejo implícito de memoria

Paradigma Declarativo - Lógico

*Los programas **consisten en definiciones de relaciones y un cómputo es una prueba (una sucesión de inferencias)**.*

programa = conjunto de axiomas o reglas lógicas, definiendo relaciones entre los objetos.

cómputo = es una deducción de consecuencias del programa.

El modelo lógico es importante porque es una **formalización del proceso del razonamiento**.

Se relaciona a las bases de datos relacionales y sistemas expertos.

La primera preocupación en la programación lógica es la *definición de las relaciones*.

Paradigma Declarativo - Lógico

- Un **programa lógico** consiste en un *conjunto de axiomas* y una *instrucción de la meta*.
- Se aplican las *reglas de inferencia* para **determinar si los axiomas son suficientes para asegurar la verdad de la instrucción de la meta**.
- La ejecución de un programa lógico corresponde a la **construcción de una prueba de la instrucción de la meta desde los axiomas**.
- El programador es responsable de especificar las relaciones lógicas básicas y no especifica la manera en que las reglas de la inferencia son aplicadas.
- Las estructuras básicas de programación lógica, heredan condiciones e instrucciones de la lógica.
- instrucciones básicas: Hechos, Reglas, Consultas.
- Hay una sola estructura del datos: el *término lógico*.

Paradigma Declarativo - Lógico

Programa: se escribe como un conjunto de axiomas que especifican las propiedades del cómputo. El resultado es la deducción de consecuencias del programa.

Propiedades

- No existe la noción de control de flujo
- Cómputo con relaciones
- Mecanismo de inferencia
- Backtracking automático

Lenguajes y proceso de desarrollo de Software

- Los Lenguajes de Programación son herramientas para construir software, razón por la cual, se debe tomar en cuenta el contexto en el cual se utilizan.
- Veremos entonces cuál es el rol de los lenguajes dentro de dicho proceso, para después entender la importancia y necesidad de su estudio.
- El software tiene un “ciclo de vida” compuesto de varias fases, las cuales independientemente del paradigma utilizado abarcan:
 - Definición: se centra en el qué.
 - Desarrollo: se centra en el cómo.
 - Mantenimiento: se centra en el cambio.

Lenguajes y proceso de desarrollo de Software

- *Actividades principales que se presentan en las fases del ciclo de vida son:*
 - ✓ Análisis de requerimientos y especificación
 - ✓ Diseño de software
 - ✓ Implementación
 - ✓ Verificación y Validación
 - ✓ Mantenimiento
- Los Lenguajes son utilizados principalmente en la *etapa de Implementación*, donde se definen los algoritmos y estructuras de datos.
- Asimismo se utilizan lenguajes en otras etapas, por ejemplo, se utilizan lenguajes de alto nivel en la fase de diseño para describir la descomposición de una aplicación en módulos.

Paradigmas con énfasis en estilo de descomposición de programas

Paradigma de programación procedural: Es el estilo convencional de programación, donde los programas se descomponen en *unidades de modularización*, que definen pasos de computación.

Paradigma de programación orientada a objetos: La programación estilo OO enfatiza la definición de *clases de objetos*. Durante la ejecución de un programa, se crean instancias de clases a medida que se necesitan. Se definen jerarquías de clases y se realiza selección en run-time de las unidades a ejecutar.

Paradigmas con énfasis en estilo de descomposición de programas

Paradigma de programación Funcional: El estilo de programación funcional se basa en la teoría de funciones matemáticas. Enfatiza el cómputo de valores por el uso de expresiones y funciones. *Las funciones son los bloques primarios de construcción;* pueden ser pasadas libremente como parámetros y pueden ser construidas y retornadas como parámetros resultado de otras funciones.

Paradigma de programación lógica: Este estilo enfatiza la descripción declarativa de un problema en lugar de la descomposición de un problema en una implementación algorítmica.

Paradigmas de LPs - Síntesis

En síntesis, se observan dos tipos de paradigmas:

- ✓ basados en el *modelo computacional* y
- ✓ basados en *los principios de organización de programas*.

Con respecto al primer tipo los paradigmas imperativo y declarativo reflejan los diferentes modelos computacionales del lenguaje.

Diferentes paradigmas de organización pueden aplicarse a diferentes modelos computacionales